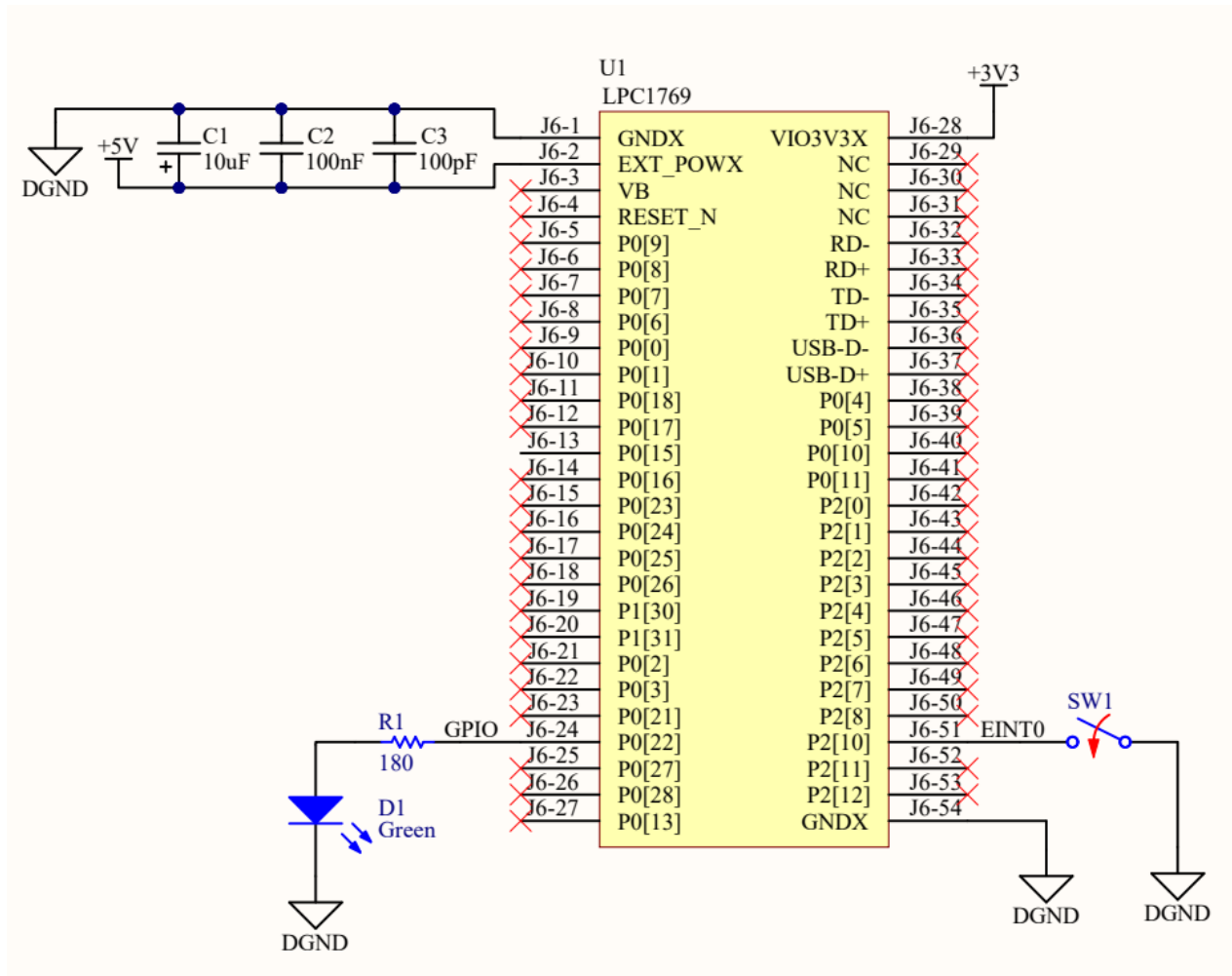


```

1  /*=====
2  * @file    Ejercicio_3_2.c
3  *
4  * @brief    Controla un LED conectado al pin P0.22, alternando periódicamente
5  *           entre los estados encendido y apagado.
6  *
7  * @details  - El tiempo de intermitencia es determinado por el estado lógico
8  *           de una entrada digital conectada al pin P2.10, vía interrupciones
9  *           externas.
10 *           - Implementar retardo bloqueante.
11 *
12 * @author   Ing. E. Migliore
13 *
14 * @version  3.0
15  =====*/

```



```

1  /*=====
2  * DIRECTIVAS DE INCLUSIÓN
3  *=====*/
4  #include "LPC17xx.h"
5
6  /*=====
7  * DEFINICIÓN DE CONSTANTES Y MACROS
8  *=====*/
9  #define NVIC_PRIO_BITS      5  /* Bits físicos del registro de prioridad:
10                                2^5 = 32 niveles de prioridad posibles
11                                (del 0 al 31)*/
12  #define EINT0_PRIORITY      1  //Prioridad 1 para Interrupción Externa EINT0
13  #define EINT0_IRQ           18 //Número de IRQ (Interrupt Request) para EINT0
14
15  #define DELAY_FAST          1000000
16  #define DELAY_SLOW          4000000
17
18  /*=====
19  * DEFINICIÓN DE TIPO DE DATOS Y VARIABLES GLOBALES
20  *=====*/
21  volatile uint32_t delayTime = DELAY_SLOW;
22
23  /*=====
24  * PROTOTIPOS DE FUNCIONES
25  *=====*/
26  void cfgGPIO(void);
27  void cfgIntExt(void);
28  static inline void cfgNVIC(int32_t, uint32_t);
29  void blinkLed(uint32_t);
30  void delay(uint32_t);
31
32  /*=====
33  * FUNCIÓN PRINCIPAL
34  *=====*/
35  int main(void)
36  {
37      /*-----
38      * INICIALIZACIÓN DEL MCU
39      *-----*/
40      cfgGPIO();
41      cfgIntExt();
42
43      /*-----
44      * INICIO PROGRAMA PRINCIPAL
45      *-----*/
46      while(1)
47      {
48          blinkLed(delayTime);
49      }
50
51      return 0;
52  }
53
54  /*=====
55  * DEFINICIONES DE FUNCIONES
56  *=====*/
57
58  /**
59   * @brief   Configura los pines P0.15 como entrada digital y P0.22 como
60   *          salida digital.
61   */
62  void cfgGPIO(void)
63  {
64      LPC_PINCON->PINSEL4 |= (1<<20);    //P2.10 EINT0 Function
65      LPC_PINCON->PINSEL1 &= ~(0b11<<12); //P0.22 GPIO Function
66      LPC_GPIO0->FIODIR  |= (1<<22);    //P0.22 Digital Output
67
68      return;
69  }

```

```

70
71 /*****
72  * @brief   Configura el NVIC y SCB para la habilitación de interrupciones.
73  *****/
74 static inline void cfgNVIC(int32_t irq_number, uint32_t priority_number)
75 {
76     //Asigna Prioridad de Interrupción
77     if(irq_number < 0) {
78         //Ejecuta interrupciones del sistema.
79         //SHP = System Handler Priority Registers
80         SCB->SHP[((uint32_t)(irq_number) & 0xF) - 4] = ((irq_number << (8 -
81             NVIC_PRIO_BITS)) & 0xFF);
82     } else {
83         //Ejecuta interrupciones de periféricos.
84         //IP = Interrupt Priority Registers
85         NVIC->IP[(uint32_t)(irq_number)] = ((priority_number << (8 - NVIC_PRIO_BITS)) &
86             0xFF);
87     }
88     //Habilitación de Interrupción
89     NVIC->ISER[((uint32_t)irq_number >> 5)] = (1 << ((uint32_t)irq_number & 0x1F));
90 }
91 /*****
92  * @brief   Configura la interrupción externa en el pin P2.10.
93  *
94  * @details - Habilita la detección de interrupción por flanco Descendente en el
95  *             pin P2.10.
96  *             - Limpia banderas pendientes previas, asigna la prioridad
97  *             en el NVIC y habilita el vector EINT0_IRQn.
98  *****/
99 void cfgIntExt(void)
100 {
101     LPC_SC->EXTMODE |= (1<<0);    //EINT0 por Flanco
102     LPC_SC->EXTPOLAR &= ~(1<<0);  //EINT0 por Flanco Descendente
103     LPC_SC->EXTINT  |= (1<<0);    //Limpia Flag EINT0
104
105     cfgNVIC(EINT0_IRQ, EINT0_PRIORITY);
106
107     return;
108 }
109
110
111 /*****
112  * @brief   Rutina de Servicio de Interrupción (ISR) para el vector EINT0.
113  *
114  * @details - Atiende las interrupciones por GPIO del Puerto 0. Verifica si la
115  *             interrupción proviene específicamente del pin P0.15 por flanco
116  *             ascendente.
117  *             - Alterna el tiempo de retardo de un LED mediante una variable `index`
118  *             y limpia la bandera de interrupción en el periférico.
119  *****/
120 void EINT0_IRQHandler(void)
121 {
122     static uint32_t index = 0;
123
124     //Selección del tiempo de retardo según el estado de index
125     index++;
126     if(index%2){
127         delayTime = DELAY_FAST;
128     }
129     else{
130         delayTime = DELAY_SLOW;
131     }
132
133     LPC_SC->EXTINT |= (1<<0);    //Limpia Flag EINT0
134
135     return;
136 }

```

```

137
138
139 /*****
140  * @brief      Ejecuta un ciclo de intermitencia del LED conectado a P0.22.
141  *****/
142 void blinkLed(uint32_t delayTime)
143 {
144     LPC_GPIO0->FIOSET |= (1<<22);    // P0.22: LED ON
145     delay(delayTime);
146     LPC_GPIO0->FIOCLR |= (1<<22);    // P0.22: LED OFF
147     delay(delayTime);
148
149     return;
150 }
151
152 /*****
153  * @brief      Genera un retardo bloqueante.
154  *
155  * @details     Ejecuta un lazo de espera durante la cantidad de iteraciones
156  *               indicada por delayTime. La duración del retardo depende del
157  *               valor recibido y de la frecuencia de CPU.
158  *****/
159 void delay(uint32_t delayTime)
160 {
161     for(volatile uint32_t i = 0; i < delayTime; i++)
162     {
163         __NOP();    // NOP: 1 ciclo de CPU
164     }
165
166     return;
167 }
168
169

```